A Two-Stage Incremental Entity Resolution Pipeline with FAISS Blocking and Splink Matching

Denis Robert

August 2026

Abstract

This paper presents and evaluates an *incremental* two-stage entity resolution pipeline: it resolves one query record at a time against a fixed reference population, the online, per-request setting of an API or a streaming insert rather than a batch deduplication job. The pipeline targets incremental (online) entity resolution — matching a new record to an existing reference as it arrives — and is not intended for one-off batch linkage of two large stored populations, where a batched linker would reconstruct the candidate graph and score it in bulk. This focus applies to incomplete Canadian person records. A synthetic reference population is represented with normalized all-MiniLM-L6-v2 embeddings and persisted in a FAISS IndexFlatIP store. Each query is first blocked to its top 20 vector neighbors, after which Splink performs interpretable probabilistic linkage using Jaro–Winkler comparisons for names and address, a dedicated email comparison, and a date-of-birth comparison that accounts for date structure. The work follows the two-stage *blocking-plus-probabilistic-matching* architecture used by dedicated entity-resolution libraries such as dedupe 2.x [6], but its blocking stage is dense FAISS retrieval over row embeddings rather than dedupe's predicate/TF-IDF canopy blocking, and it was independently derived with a different implementation based on off-the-shelf components and evaluation design. In a 40,013-query experiment over identical records, six clerical perturbation types, close-variation records, and unrelated records, the pipeline achieved 99.936% precision, 98.29% recall, and a 99.10% F1 score (precision is a near-best-case bound: the synthetic negatives are generated to share no full identity signature with their reference record); the evaluation is strict (a positive counts only when matched to its own reference row) and the query deck is shared with the Section 7 serialization and threshold sweeps for stage-by-stage comparability. Query latency is reported as the *cold, per-query* time of the production resolver on the 50,000-record index: a median ≈ 45 ms end-to-end (mean ≈ 45 ms, $p95 \approx 56$ ms; Section 8), of which the trained lightweight scorer contributes roughly 15 ms; this replaces the previous per-query Splink `Linker` construction and reduces the earlier ≈ 0.4 s median by an order of magnitude. The pipeline scores with Splink's untrained default m/u and a fixed prior of 10^{-4}, so the decision thresholds are operating-score cut-offs rather than calibrated posterior probabilities. Deployment concerns — memory scaling, operational limits, parameter calibration, and the choice of an approximate index or external vector store — are treated as design levers for the operator's latency and cost profile rather than as fixed results.

1 Introduction

Entity resolution attempts to determine whether two records refer to the same real-world entity. This task is difficult when records contain typographical variation, missing values, stale addresses, or inconsistent contact information. Comparing every query against every reference record is also unnecessarily expensive: for N reference records, exhaustive comparison requires $O(N)$ candidate comparisons per query.

The implementation in the entity-resolution repository addresses this problem with a retrieval-and-linkage architecture. It is an *incremental* resolver: it matches a single incoming query record against a fixed reference population on demand, which is the online, per-request setting of an API or a streaming insert. It is not a batch entity-resolution engine; linking two large stored populations in bulk is out of scope, because the resolver builds and scores top-k candidates per query rather than materialising and clustering all pairwise comparisons at once. A dense vector index [9] performs approximate or exact nearest-neighbor retrieval, reducing the candidate set to the closest k records. This use of dense retrieval for blocking follows a line of work including DeepER [5], pre-trained embedding comparisons [4], and universal dense blocking [3]. Splink then evaluates those candidates with explicit field comparisons and returns a match only when its probability exceeds a configurable threshold.

The representation choice is informed by recent work comparing tabular representation approaches. Vogel et al.'s *Towards Universal Tabular Embeddings: A Benchmark Across Data Tasks* (arXiv:2604.21696v1, `https://arxiv.org/abs/2604.21696v1`) [8] benchmark embeddings at cell, row, column, and table levels and show that performance depends on the downstream task and representation level; in the relevant current evaluation, textual embeddings outperform the dedicated tabular alternatives for retrieval. Franz et al.'s *Universal Embeddings of Tabular Data* (arXiv:2507.05904v1, `https://arxiv.org/abs/2507.05904v1`) [7] describes a different strategy that transforms tabular data into a graph, learns entity embeddings with graph auto-encoders, and aggregates them into row embeddings. This graph-based approach may bear fruit for entity resolution in future experiments because it is designed to capture relationships among tabular entities and to embed unseen samples composed of similar entities. The present system therefore uses a row-level textual representation as its current baseline while leaving room to replace the embedding engine as these alternatives become more capable.

2 System Objective and Data Model

The reference population contains 50,000 synthetic Canadian person records. Each record has:

- first name;
- last name;
- date of birth;
- a Canadian address; and
- an email address.

Pipeline architecture. Figure 1 gives the central view of the pipeline that the rest of the paper evaluates stage by stage. It is an *incremental*, two-stage resolver over a fixed reference population: the reference is embedded once and persisted as a normalized FAISS index (with the person metadata alongside), then every incoming query follows the same per-request path — embed the query with the same model, retrieve the top-k candidates by cosine similarity, score those candidates with the Fellegi–Sunter matcher, and emit the match decision under the operating threshold. Two properties of this flow recur throughout the paper. First, the reference index and its metadata are built offline and load without re-embedding, so the online path never touches the bulk of the data. Second, the matcher is a lightweight scorer over the trained comparison weights (trained once with Splink, served at query time by custom code that reuses Splink's comparison

definitions), which is what keeps the per-request cost to tens of milliseconds rather than a per-query Splink `Linker` construction.

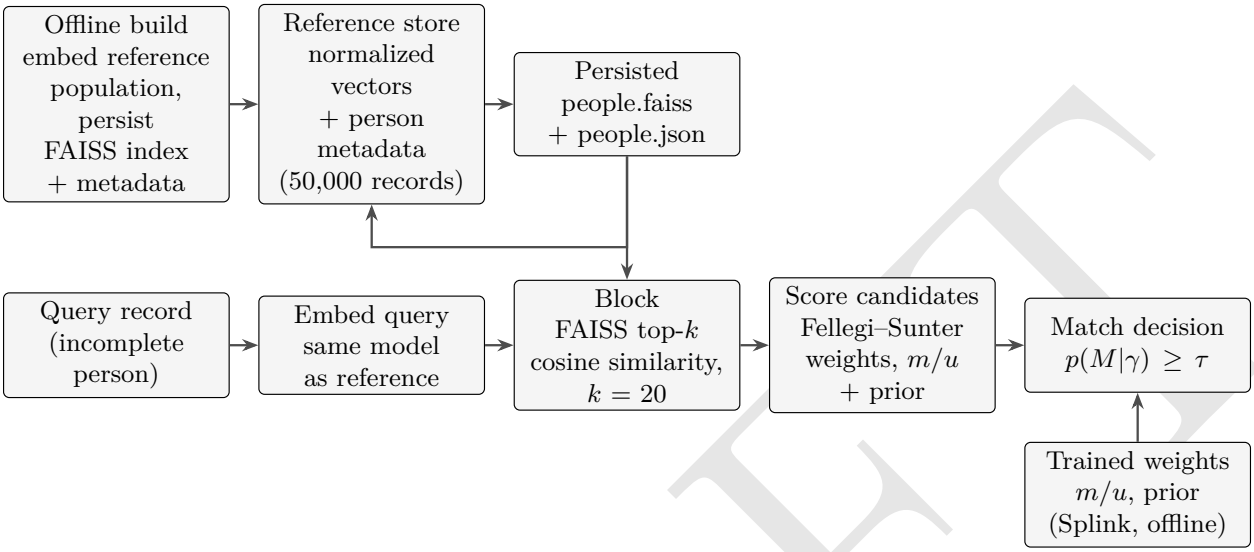

Figure 1: Pipeline architecture: an offline build embeds the reference population once and persists the vector index plus metadata; each incremental query embeds with the same model, blocks to the top-k cosine neighbours, scores them with the trained Fellegi–Sunter weights, and emits the thresholded match decision. The trained weights come from a one-time Splink pass reused at query time by the lightweight scorer.

Synthetic-data generator. The population is produced by the project's `generate_data.py` with a fixed seed (42). Given names are gender-correlated: each record is drawn male with probability 0.498 (Statistics Canada 2021 Census share) and the given name is sampled from the male or female Faker name pool accordingly. The province or territory is sampled weighted by its 2021 Census resident population, so Ontario (38%), Quebec (23%), British Columbia (14%), and Alberta (12%) dominate as in the real national distribution. Addresses are Faker Canadian addresses whose postal-code forward-sortation-area prefix letter is drawn from the Canada Post mapping for that province (e.g. K–P for Ontario, G/H/J for Quebec, V for British Columbia), so the code and the province in a record always agree. Emails are Faker `ascii_safe_email()` addresses (varied local parts and example domains) rather than a mechanical name-based pattern, and 30% of addresses and emails are independently omitted. Last names are drawn from Faker's en_CA surname pool; a non-English given-name distribution is left as future work.

Address and email are independently removed with a configurable default probability of 0.30. Consequently, first name, last name, and date of birth are the most consistently available identity attributes, while address and email are useful but incomplete evidence. Addresses are also treated as less stable because people move and formatting conventions change.

A note on preprocessing: the problem that inspired this research supplied person records with pre-parsed first and last name fields and an address field already standardized against a postal reference database, so the pipeline required no name or address segmentation and no normalization preprocessing before embedding and linkage; it consumes structured fields as given. The relaxation of that assumption, and its accuracy and resource implications, are examined in the parsing-and-segmentation research item of Section 10.6.

A record is serialized into a structured text string:

```
First Name: ...
Last Name: ...
Date of Birth: ...
Address: ...
Email: ...
```

Missing fields are omitted from the embedding text and retained as null metadata. The same person object is used for the query and for the persisted metadata, avoiding a second source of truth.

2.1 Row serialization strategies

The embedding ablation compares three textual representations of the same structured record. These representations affect only dense retrieval; Splink receives the original structured fields and uses the same comparisons in every experiment. For example, consider the record

```
first_name    = Alice
last_name     = Wong
date_of_birth = 1985-06-15
address       = 123 Main Street, Toronto, ON M5V 1A1
email         = alice.wong@example.com
```

The **default** strategy is the production representation implemented by `Person.to_text`. It uses labelled lines in identity, address, and email order:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
Address: 123 Main Street, Toronto, ON M5V 1A1
Email: alice.wong@example.com
```

The **identity-first** strategy retains the same labels and values but places the optional contact fields in email-then-address order after the identity fields:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15
Email: alice.wong@example.com
Address: 123 Main Street, Toronto, ON M5V 1A1
```

Both labelled strategies place the invariant identity fields (first name, last name, date of birth) first and label them explicitly; the **identity-first** manipulation is the ordering of the optional contact fields, which are listed email-then-address instead of address-then-email. This tests whether the relative position of the contact fields in the row text changes retrieval, without changing the vocabulary, the field labels, or the underlying data: The **compact** strategy removes labels and joins the fixed field sequence with pipe delimiters:

```
Alice|Wong|1985-06-15|123 Main Street, Toronto, ON M5V 1A1|alice.wong@example.com
```

For a record with missing address and email, the labelled strategies omit those lines, whereas compact serialization preserves field positions with empty values:

```
First Name: Alice
Last Name: Wong
Date of Birth: 1985-06-15

Alice|Wong|1985-06-15||
```

The default strategy is the baseline used by the resolver and confusion-matrix experiment. Identity-first and compact are evaluation alternatives, not separate linkage models. Comparing them under the same FAISS index type, blocking sizes, Splink configuration, queries, and thresholds isolates the effect of row representation on candidate recall and downstream matching.

3 Two-Stage Algorithm

3.1 Offline generation and indexing

The offline process in `generate_data.py` performs the following steps:

1. Generate the reference population with a fixed random seed when reproducibility is required.
2. Convert every person into its textual row representation.
3. Compute a dense vector with `sentence-transformers/all-MiniLM-L6-v2` through `HuggingFaceEmbeddings`.
4. L2-normalize each vector and add it to a FAISS `IndexFlatIP` index. Inner product on normalized vectors is cosine similarity.
5. Persist the FAISS index as `people.faiss` and the person records, model name, and normalization setting as `people.json`.

The metadata file is deliberately separate from the FAISS binary index. FAISS stores the vectors and their positional order; the JSON file stores the record associated with each position. Loading reconstructs the embedding client and reuses the saved vectors without re-embedding the reference population.

3.2 Online blocking

Given a query record q, the online (incremental) process embeds the same textual representation and searches the index for the top k records; each query is handled independently and immediately, without rebuilding or re-clustering the reference population (the defining incremental property; see Figure 1). The default is $k = 20$. If $e(q)$ and $e(r)$ are normalized query and reference vectors, the blocking score is

$$s_{\text{FAISS}}(q, r) = e(q)^T e(r),$$

which is cosine similarity in the range $[-1, 1]$. This stage is optimized for recall: the true match must be included in the candidate set for the second stage to recover it. FAISS [9] provides this nearest-neighbour search.

3.3 Temporal-aware blocking for records with known capture dates

The blocking score above weights every retrieved reference equally. When reference records carry a known capture (creation) date, the relevance of each field to the match decision should be weighted by the age gap between records. The continuous decay model used to age address evidence at retrieval time — equation (1) — together with the full discussion of address volatility, both decay formulations, and their measured impact, is developed in Section 3.4; this subsection describes how that fused score is evaluated over the index and reports the temporal experiment.

The implementation does not evaluate this fused score over every reference record. Each sub-index (identity, address) is searched only to depth $K = \max(k, 32)$ and the fused score is computed over the union of the two retrieved pools, followed by a top-k selection. Eq. (1) is therefore the *re-ranking rule over that finite pool*, not an exact global re-ranking: recall is capped before fusion by what each sub-index retrieved at depth K, and enlarging K narrows the gap to full re-ranking at higher retrieval cost.

Measured on the synthetic population with a controlled age gap (6,000-record index, 180 duplicate pairs, T=10 years; artifact `results/erwhitepaper/temporal_gap_results.json`, produced by `experiment_temporal_gap.py` with seed 42 and views `full identity contact multi_union gap_weighted two_tier`), this recovers the temporal failure mode: in the long-gap cohort the address-only "contact" view collapses to 0.33 recall at k=1 (cohort-stratified 0.33), while the gap-weighted score recovers 0.96 at k=1 and 1.0 by k=5 — comparable to relying on invariant identity alone (1.0 at all k), and far above the unweighted single view. In the short-gap cohort the gap-weighted view also reaches 1.0 by k=5, at the cost of a k=1 penalty against the address-rich "full" view (0.93 versus 1.00 short-cohort recall), which disappears in the working range ($k \geq 5$). The full table and the companion analysis appear in the batch-deduplication study [26], and the persisted artifact records the exact command, seed, and library versions. Two scope notes apply: this is a synthetic controlled stress test — long-gap queries replace the address with a freshly generated one while preserving all identity fields — rather than an observation of real residency dynamics, and T=10 years is an experimental setting, not an empirically estimated residency parameter. When capture dates are unavailable, or when the population is effectively co-temporal (all records sharing one generation date, as in the synthetic experiments of Section 8), the temporal caveat does not apply and the single-vector "full" index suffices. The decay rate T must be set from the deployment's observed residency distribution rather than assumed.

A note on stage: the decay in (1) acts at *blocking* time — it attenuates a cosine *retrieval* score during candidate selection and leaves the comparison model time-independent. This stage distinction, the linkage-side (comparison-level) formulation, and the relationship to the temporal record-linkage literature are discussed in Section 3.4.

3.4 Address volatility and the temporal decay of evidence

The address is the field whose evidentiary value is genuinely time-varying, and its volatility surfaces in several ways across this pipeline. Names change only in controlled ways (nicknames, abbreviations, marriage) and date of birth is invariant, so these form reliable long-gap anchors; email is abandoned or rotated but leaves little persistent trace. The address, by contrast, is stable within a short window — two records captured near each other at the same address are strong evidence of the same entity — but becomes actively misleading once records are more than roughly the typical residency period apart, when the entity has since moved. Because address evidence changes over time, its weight must account for the age gap between records at *every* stage where it is used: in the retrieval score, in the comparison model, and in how aggressively the address should be trusted.

This subsection consolidates that discussion.

Continuous decay at retrieval time. For a query q and reference r, let $\Delta t_{q,r}$ be their capture-date gap and T the residency timescale of the deployment. The multi-index blocker ages each field's retrieval score continuously rather than applying a single fixed weight:

$$s_{\text{block}}(q, r) = s_{\text{identity}}(q, r) + e^{-\Delta t_{q,r}/T}\, s_{\text{address}}(q, r), \tag{1}$$

where s_{identity} is the retrieval score on the invariant fields (first, last, date of birth) and s_{address} the score on the address view. The identity score always counts at full weight; the address score is exponentially down-weighted as records age apart. Because $\Delta t_{q,r}$ is a pairwise quantity, the weight is applied at *score-fusion time* (after retrieving each view) rather than baked into a precomputed vector. The mechanism that evaluates this score over a finite pool, and the measured temporal experiment, are given in Section 3.3.

Comparison-level (linkage-side) decay. The same decay admits a second, piecewise formulation inside the comparison model. As Robin Linacre notes in a public comment on Splink issue #3240 [25], the address comparison can be bucketed by the capture-date gap into comparison levels, such as "same address, capture dates at most one year apart," "same address, one to five years apart," "same address, five to ten years apart," "same address, more than ten years apart," and "different address." Each level carries its own m/u probabilities and therefore its own match weight, and address disagreement can be stratified by time gap in the same way. This yields a piecewise approximation to the continuous decay of (1) without adding a weighting mechanism to the matcher and without committing to a specific parametric decay function — at the cost of committing to a fixed set of bucket boundaries. The fidelity of the approximation is set by the bucket widths: refining them moves the step function toward the smooth entropy-loss weight $w_c(\Delta t)$ derived in the companion literature survey [24], at the price of needing enough training or estimated pairs in every bucket for its m/u to be stable. The measured pipeline in this whitepaper uses the retrieval-time fusion of (1) and keeps the comparison model time-independent, but deployments already estimating per-level m/u can obtain the same qualitative effect by the bucketing route. The temporal record-linkage literature applies decay to the comparison weights instead of the retrieval score; why the blocking-side decay cannot simply be absorbed into m/u is treated in the companion design note [25].

Measured impact: two complementary probes. Two experiments probe, from opposite directions, when decaying address evidence helps. The temporal stress test (Section 3.3, measured with a controlled age gap, T=10 years experimental) shows the failure mode time-decay fixes: in the long-gap cohort the address-only "contact" view collapses to 0.33 recall at k=1, while the gap-weighted score recovers 0.96 at k=1 and 1.0 by k=5 — comparable to relying on invariant identity alone. The complementary address-"weakening" sweep (Section 8.4) asks whether the address is so volatile that it should be down-weighted *unconditionally*: on the synthetic Canadian population it is not, because 30% of addresses are missing independently and the addresses that exist are long and highly discriminative, so agreement on the address rescues close same-entity queries — weakening it has no effect on the reduced-scale address sweep (F1 99.64% at every strength; artifact `f1_address_sweep_results.json`). The first probe says address decay matters when capture dates and genuine gaps are present; the second says the address is too informative in the co-temporal synthetic corpus to discount outright. Together they frame the operative question as *when* (conditioned on capture-date gap) rather than *whether* to trust address evidence.

When it applies. The decay model of (1) requires records with known capture dates and a population that is not effectively co-temporal; for the co-temporal synthetic population (all records sharing one generation date, as in the confusion-matrix experiments of Section 8), the temporal caveat does not apply and a single-vector "full" index suffices. The residency timescale T must be set from the deployment's observed residency distribution rather than assumed. Whether weakening a volatile address field is ever beneficial — for example, addresses that change frequently between legitimate records of the same entity, noisy or generic addresses (unit-only entries, post-office boxes), or populations with many address collisions in high-density buildings — is an open, data-dependent question requiring controlled experiments across data sets with tunable address volatility; `experiment_f1_sweep.py` and `weaken_comparison` make such a study straightforward, and real-address volatility is probed on the NC-voter data in Section 9 and flagged for drift monitoring in Section 10.

3.5 Probabilistic linkage

The candidate set is passed to Splink as a small deduplication problem containing the query and the retrieved records. Splink implements a scalable form of probabilistic linkage grounded in the Fellegi–Sunter framework [1, 2]. The configured comparisons and their intended evidence priority are listed in Table 1:

Table 1: Field comparisons and intended evidence priority.

Field	Comparison	Intended role
First name	Jaro–Winkler thresholds	High-priority identity evidence
Last name	Jaro–Winkler thresholds	High-priority identity evidence
Date of birth	Date-of-birth comparison	Strong, stable evidence
Email	Email comparison	Medium-priority evidence when present
Address	Jaro–Winkler thresholds	Lower-priority, *changeable* evidence (see Section 3.4)

Splink produces a match probability from the comparison vector and its m/u parameters. The resolver selects the highest-scoring candidate involving the query and returns it only when

$$P(\text{match} \mid \text{comparisons}) \geq \tau,$$

where τ is the configured match threshold. Missing values are not treated as negative evidence; they provide no comparison agreement and the remaining fields determine the result.

Operating score versus calibrated probability. Splink's output is a posterior-like *operating score* built from the configured m/u weights and the match prior. Its numerical value is a calibrated probability — and quantities such as $\tau^* = \frac{C_{FP}}{C_{FP}+C_{FN}}$ (Appendix A) are decision-theoretically meaningful — only when those m/u values and the prior are fitted to the deployment population. The results in this paper use Splink's untrained default m/u and a fixed prior of 10^{-4}, so the scores give a principled *ranking* and thresholded operating points; they are not presented as calibrated posterior probabilities of latent ground truth, and the reported thresholds (0.85, 0.95) should be read as operating-score cut-offs for the comparison task, not as cost-derived Bayes decisions. Calibrating m/u on operational labelled pairs, and re-anchoring the prior and τ jointly, is precisely the future-work agenda of Sections 8.2 and B.

The implementation uses FAISS for blocking, Splink 4.x for training the m/u parameters, and a lightweight scorer for the per-query linkage (the train-with-Splink, infer-with-custom-code architecture of the next paragraph). The current settings provide comparison ordering and thresholded similarity levels. Ideally, in a production deployment, Splink's m/u probabilities should be trained on representative labelled pairs rather than relying on defaults.

Linker architecture: train with Splink, infer with a lightweight scorer. In this pipeline Splink is used as a *training* engine, not as the per-query inference engine. Training — estimating u by random sampling, the match prior, and m by expectation–maximisation (optionally on labelled or sampled pairs) — is performed in a batched Splink `Linker` over the reference population, and the resulting per-comparison, per-level m/u (and prior) are persisted to a model settings file. At query time the resolver does *not* construct a Splink `Linker`: it loads those weights into a small, purpose-built scorer that reproduces Splink's match-score definition exactly. For each candidate the scorer assigns, for every field, the level whose Splink-generated `sql_condition` the pair satisfies (using the very comparison objects Splink's `predict` lowers to SQL, so the value-to-level mapping is identical by construction), multiplies the level bayes factors m_i/u_i with the prior odds $p/(1-p)$, and converts to a probability by $P = BF/(1+BF)$. All of a query's candidates are scored in a single vectorised pass over one shared connection, and the comparison `CASE` expressions are precompiled once. This removes the largest component of the old per-query latency — constructing a fresh Splink `Linker`, DuckDB backend, and pipeline per request — and makes the trained m/u directly reusable, since they are plain persisted numbers rather than state trapped inside a `Linker` object. The scorer reproduces Splink's inference to numeric precision (mean absolute posterior difference $\approx 10^{-7}$ over real candidate pairs), so the resolver is drop-in equivalent to the per-query Splink path it replaces while cutting end-to-end query latency by roughly an order of magnitude (Section 8).

A note on the linkage side of the decay, which is now treated in Section 3.4: the temporal decay of address evidence can also be expressed as bucketed comparison levels inside the matcher (each capture-date gap band carrying its own m/u), a piecewise alternative to the continuous retrieval-time decay of (1). The measured pipeline uses the retrieval-time fusion of (1) and keeps the linkage model time-independent.

4 Persistence and Operational Workflow

The basic workflow. The pipeline is operated with two explicit commands, an offline build and an online query:

```
python scripts/generate_data.py --count 50000 --missing-rate 0.3 --output-dir data
python scripts/search_cli.py --index-dir data --input-json query.json --threshold 0.85
```

The first command is the offline build step of Section 3.1. It generates a reference population of 50,000 synthetic Canadian person records with address and email independently missing at a default rate of 0.3 — serializes every record into the textual row representation of Section 2.2, embeds the rows with `sentence-transformers/all-MiniLM-L6-v2` through `HuggingFaceEmbeddings`, L2-normalizes the vectors, and adds them to a FAISS `IndexFlatIP` index. The artifact is persisted as two files: the binary index `people.faiss`, which holds the normalized vectors in positional order, and `people.json`, which stores the person record, model name, and normalization setting associated with each position. Keeping the files separate is deliberate (Section 3.1): the vector store answers nearest-neighbour queries, the metadata file supplies the record behind every position, and a reload reuses the saved vectors without re-embedding the reference population.

The second command is the online query path. It loads the persisted pair of files, reconstructs the embedding client from the saved model name, and reuses the stored vectors without re-embedding the reference population. Given an input person in `query.json`, it applies the same textual row representation used at build time, embeds the query record, and searches the index for the top 20 candidates under the default block size (Section 3.2). The candidate set is passed to Splink as a small deduplication problem of one query against its retrieved records, using the comparisons listed in Table 1; the command then either emits a match — the highest-scoring candidate together with its Splink probability and the matched metadata — when that probability reaches the configured threshold τ, or a no-match decision when it does not. Separating the two commands prevents the expensive population embedding and indexing from being repeated on every query and makes the persisted index a deployable artifact.

5 FAISS Applicability and Memory Bounds

The 50,000-record artifact generated by this implementation provides a concrete sizing baseline. On disk, the measured `people.faiss` file is 76.8 MB and the `people.json` metadata file is 10.5 MB, for a combined 87.3 MB. This corresponds to approximately 1,536 bytes per normalized 384-dimensional float32 vector, 210 bytes per serialized person record, and 1,746 bytes per record including both artifacts. The index dimension follows the MiniLM embedding model used by the baseline.

The following extrapolation assumes the same embedding dimension, float32 storage, flat inner-product index, average metadata length, and one metadata record per vector. It uses binary RAM units (1 GiB = 2^{30} bytes), although the table labels them as the commonly used 16, 32, and 128 GB deployment sizes. It excludes the Python interpreter, embedding model, temporary construction buffers, allocator fragmentation, operating-system memory, and concurrent queries. Table 2 summarises the resulting capacity bounds.

Table 2: Estimated capacity when the FAISS index and person metadata share the available RAM.

Available RAM	Raw capacity	Index size	Metadata size	Recommended*
16 GB	9.84 million	14.4 GB	2.0 GB	6.89 million
32 GB	19.68 million	28.9 GB	4.1 GB	13.77 million
128 GB	78.71 million	115.3 GB	16.5 GB	55.10 million

*Recommended capacity reserves 30% of RAM for the operating system, Python process, model weights, FAISS temporary allocations, and query concurrency. The raw capacity is a sizing upper bound, not a production recommendation. During generation, peak memory can be higher because embeddings may be materialized before being added to FAISS. Building large indexes should therefore use batches and measure peak resident set size rather than relying only on final file size.

For this flat `IndexFlatIP` design, a query scans all vectors, so memory capacity and query latency are coupled to the record count. At tens of millions of records, an approximate index such as HNSW or an inverted-file/product-quantized index should be evaluated; FAISS [9] ships several such indexes. Quantization can reduce memory substantially, but may reduce blocking recall and must be measured against the downstream Splink match rate. Regardless of the FAISS index type, the metadata mapping must preserve vector-to-person row order and should be loaded with equivalent memory budgeting.

Metadata sizing caveat. The table's metadata column scales the measured serialized JSON bytes

linearly with record count. It does not include Python object overhead, columnar representation, or the deserialized in-memory model (dicts, strings, and the documents list used by the store), which can dominate the serialized size at scale. The 1,746 bytes/record figure should therefore be treated as a planning lower bound on resident-set size, not a measured memory ceiling; production sizing should measure peak RSS at target scale rather than rely on file sizes.

5.1 When an external vector database becomes appropriate

An external vector database is not automatically better than FAISS. For a single service instance, a stable reference population, and a dataset that fits comfortably in memory, local FAISS has fewer moving parts and avoids network latency. The decision should be based on operational bounds as well as raw capacity. With the measured baseline, 10 million records would require approximately 17.5 GB for the flat index plus metadata, while 50 million records would require approximately 87.3 GB before runtime overhead. These sizes place a single-process deployment near or beyond the recommended envelope of a 16 or 32 GB host and make a 128 GB host increasingly specialized.

An external vector database is likely to become more appropriate under any of the following conditions:

- the working set approaches roughly 70% of host RAM, leaving insufficient room for the embedding model, Python, Splink, index rebuilds, and concurrent queries;
- the reference population is large enough that flat-index scan latency or index-build time violates the service-level objective, even after testing FAISS approximate indexes;
- multiple application instances need a shared index, replicas, rolling updates, or failover without copying a large binary artifact to every host;
- records require frequent inserts, deletes, or metadata updates, making immutable local snapshots operationally expensive; or
- the system needs managed durability, access control, monitoring, backups, multi-tenant isolation, or independent scaling of storage and query capacity.

As a practical boundary, local FAISS is a strong default for the 50,000-record baseline and remains straightforward through the low millions. Between roughly 7 million and 14 million records, the 16/32 GB sizing envelope suggests benchmarking an approximate FAISS index against a managed or self-hosted vector database. Above roughly 50 million records, or whenever the index must be shared across services and regions, an external system is generally the safer operational choice.

These figures are illustrative capacity-envelope estimates, not empirical findings. They are derived from the storage arithmetic of one embedding model (384-dimensional float32), one metadata format, and the flat `IndexFlatIP` implementation of this paper, with 30% host RAM reserved for runtime overhead; they were not measured for latency, concurrency, rebuild, replication, or operational behaviour at those scales. Compression, sharding, hardware, and the host memory ceiling can move them substantially, and a strict latency or availability requirement can move the decision earlier. The external database should still be evaluated by blocking recall at $k = 20$, not only by vector-query latency, because Splink cannot recover a true match that the retrieval layer omits.

6 Complexity and Failure Modes

Let d be the embedding dimension, N the reference population size, and k the blocking size. Building the flat FAISS index costs approximately $O(Nd)$ vector storage and embedding work.

A flat inner-product query costs $O(Nd)$, although FAISS offers approximate indexes when the population grows beyond the intended scale. Splink then operates on only $k+1$ records, making the linkage stage independent of the full population size after blocking.

The principal failure mode is blocking recall. A correct record omitted from the top k candidates cannot be recovered by Splink. Other risks include embedding representations that overemphasize address tokens, duplicate or common names, poorly calibrated Splink probabilities, and distribution shift between synthetic data and operational data. Evaluation should therefore report blocking recall separately from final precision, recall, and calibration.

7 Evaluation Plan

A useful benchmark should contain labelled positive and negative pairs, controlled missingness, realistic address changes, spelling errors, and hard negatives sharing names or dates. The following metrics should be reported:

1. blocking recall at $k \in \{10, 20, 50, 100\}$;
2. final precision, recall, F1, and false match rate at several thresholds;
3. precision–recall and calibration curves for Splink probabilities;
4. mean and tail query latency; and
5. index size and offline build time.

The benchmark should include ablations for missing email, missing address, both fields missing, address changes, and name perturbations. It should also compare row serialization strategies so that improvements are attributable to representations rather than only to the matcher.

7.1 Measured Section 7 benchmark

These requirements were implemented in `experiment_section7_eval.py` and run with 5,000 reference records, the perturbed query deck of Section 8 (identical + six clerical perturbation kinds + close + unrelated per row; 40,013 queries, of which 35,013 are positives), a 30% independent missingness rate, and 250 records per ablation. Splink used its current untrained default m/u parameters, so the measurements are an engineering baseline rather than calibrated production probabilities. The evaluator wrote the complete results to `results/erwhitepaper/section7_results.json` and `results/erwhitepaper/section7_metrics.csv`.

Table 3 compares the three row serialization strategies. Blocking recall is measured over the 35,013 positive queries. Final precision, recall, and F1 use the strict definition of Section 8: a positive query is a hit only when Splink matched it to its *own* reference row. Final linkage metrics use the production top-20 candidate set, while recall is also reported at $k = 10, 50, 100$. Latency percentiles cover embedding plus FAISS blocking per query; the reported end-to-end mean adds the batched Splink inference time divided by the number of queries.

The compact serialization was the strongest of the three in this run, improving top-20 blocking recall from 99.61% to 99.76% and F1 at threshold 0.85 from 99.104% to 99.175% (the false-match rate is 0.44% for default and 0.38% for compact here). The difference is confirmed at reduced scale by the repeated-seed companion (`experiment_section7_repeated.py`, five seeds at n=1500; artifact `results/erwhitepaper/section7_repeated_results.json`): compact averages

Table 3: Measured serialization and retrieval/linkage results on the perturbed 40,013-query deck (artifact `results/erwhitepaper/section7_results.json`).

Strategy	R@10	R@20	R@50	R@100	$P_{.85}$	$R_{.85}$	$F1_{.85}$
Default	99.37%	99.61%	99.75%	99.81%	99.936%	98.29%	99.104%
Identity first	99.38%	99.62%	99.73%	99.80%	99.933%	98.29%	99.104%
Compact	99.61%	99.76%	99.85%	99.90%	99.942%	98.42%	99.175%

F1 $99.262 \pm 0.030\%$ and R@20 99.888% versus default $99.205 \pm 0.036\%$ and 99.762% (mean differences +0.057 F1 and +0.126 R@20, both outside the inter-seed spread), under the term-frequency-adjusted scorer. Serialization is thus a small but reproducible retrieval lever on this population. For the default strategy, index build time was 50.7 seconds, the mean per-query blocking (embedding + FAISS) latency was 9.2 ms, and the batched estimated mean end-to-end latency was 42.2 ms. The corresponding values were 43.8 seconds, 8.9 ms, and 41.8 ms for identity-first, and 35.4 seconds, 7.1 ms, and 41.9 ms for compact. These are batched-throughput figures (the per-query online path is measured separately in Section 8). Because the evaluator embeds and searches the whole deck as a batch, it reports a single mean per-query latency rather than a measured percentile spread.

Table 4: Blocking-recall ablations at $n = 250$ per condition (artifact `results/erwhitepaper/section7_results.json`).

Condition	R@10	R@20	R@50	R@100
Baseline	100.0%	100.0%	100.0%	100.0%
Missing email	100.0%	100.0%	100.0%	100.0%
Missing address	99.6%	100.0%	100.0%	100.0%
Missing both	98.4%	99.6%	99.6%	100.0%
Address change	100.0%	100.0%	100.0%	100.0%
Name perturbation	99.6%	100.0%	100.0%	100.0%

Table 4 summarises the blocking-recall ablations. At the default 0.85 threshold, every ablation query was accepted in this synthetic positive-only slice once it reached Splink, so the main observed degradation was retrieval: removing both optional fields reduced top-10 recall to 98.4%, recovering to 99.6% by top 20 and remaining at 99.6% at top 50 (100% by top 100); removing only the address also lowered top-10 recall to 99.6%. Missing email, address changes, and name perturbations were recovered perfectly at every k. The evaluator also produces ten-bin reliability data and threshold curves; those outputs should be recalculated after Splink is trained on labelled pairs.

8 Confusion-Matrix Experiment

The implementation includes `experiment_confusion_matrix.py`, which evaluates the current algorithm on $n = 5{,}000$ generated reference rows. The query deck is built from exactly the clerical errors a real deployment must tolerate (Section 8.1): for every reference person the test creates an identical row labelled as the same entity, one positive query for each of the six `PersonPerturbator` error types the record supports (initialled first name; a typo in first/last name or date of birth; an address typo; address denormalisation; an email typo; removal of the address or email), a close but non-identical row generated with controlled perturbations and labelled as the same entity, and an independently generated unrelated row labelled as a different entity that is not present in the reference index. A perturbation type is emitted for a row only when the record carries the field

it needs (e.g. a record without an email has no email-typo query), and every positive query is a guaranteed-different duplicate of its base row. FAISS retrieves the top 20 candidates for every query, after which Splink applies the same field comparisons and decision threshold used by the resolver. The experiment batches the queries into one Splink link-only inference call for execution efficiency; this changes execution strategy but not the candidate pairs or scoring configuration. The experiment's metaparameters are listed in Table 5. The same deck, built by the shared builder `perturbed_case_tuples`, is reused by the serialization benchmark of Section 8 and the threshold/address sweep so the three experiments measure the identical harder population.

Table 5: Confusion-matrix experiment metaparameters.

Parameter	Value
Reference/test rows (n)	5,000
Queries per test row	≈ 8 (identical + applicable perturbations + close + unrelated)
Total queries	40,013
Missing address/email rate	0.30
Embedding model	`all-MiniLM-L6-v2`
FAISS blocking size (k)	20
Splink match threshold (τ)	0.85
Close-row variation rate	0.15
Random seed	42

Table 6: Measured confusion matrix over 40,013 queries (artifact `results/erwhitepaper/confusion_matrix_results.json`).

	Predicted match	Predicted no match
Actual same entity	34,413 (TP)	600 (FN)
Actual different entity	22 (FP)	4,978 (TN)

Table 6 summarises the measured confusion matrix. All 5,000 identical queries were correctly matched. The failures concentrate in the structural perturbation kinds the old "close" row did not exercise: initialled first name (R=90.5%), the legacy close variant (97.6%), and identity typos (99.7%), while address and email perturbations and missing-field variants are recovered essentially perfectly through the full pipeline. Among the 5,000 unrelated queries, 22 were incorrectly accepted and 4,978 were rejected. The resulting precision was 99.94%, recall 98.29%, and F1 99.10%, measured with the strict per-row definition of Section 8 throughout.

Per-kind through-pipeline results. Because the deck labels each positive query by its perturbation kind, the same run reports recall through the *entire* pipeline (blocking + linkage + threshold) per kind, not only at the blocking stage (Table 7). This is the headline result of the harder deck: the two-stage pipeline is nearly perfect for address- and email-level noise, and its remaining errors are concentrated in name-level changes (initialled first name, identity typos) and the legacy close-variant fallback. The per-kind ordering closely tracks the blocking-recall ordering of Section 8.1; the matcher recovers only a small fraction of the blocked-but-not-top-ranked candidates, so the retrieval stage remains the binding constraint for name-level noise.

Table 7: Through-pipeline recall by perturbation kind on the 5,000-row / 40,013-query deck ($\tau = 0.85$, $k = 20$; artifact `results/erwhitepaper/confusion_matrix_results.json`). "Queries" reflects field-eligible emission (rows lacking the perturbed field emit no query). Blocking recall is the same-query-set top-20 indicator recorded in the artifact; linkage recall is the strict TP rate (query matched its own reference at τ).

Kind	Queries	Blocking recall	Linkage recall
Identical	5,000	100.0%	100.0%
Initialled first name	5,000	99.7%	90.5%
Typo in identity (name or DOB)	5,000	99.9%	99.8%
Typo in address	3,490	100.0%	100.0%
Address denormalisation	3,490	100.0%	100.0%
Typo in email	3,495	100.0%	100.0%
Missing address or email	4,538	99.9%	99.9%
Close same-entity (legacy close)	5,000	97.7%	97.6%
Overall	40,013	99.61%	98.29%

Metrics reported. Following the entity-resolution reporting guidance of Christen [23], we report precision, recall, and F1 rather than accuracy or specificity. In a link-only evaluation the candidate deck is dominated by true non-matches, so accuracy and specificity are driven by the majority class and are uninformative for comparing linkage configurations; precision, recall, and F1 — all computable directly from the confusion-matrix counts in Table 6 — are the quantities that reflect linkage quality at the operating threshold. The same convention is used throughout.

Term-frequency adjustment. Splink's email comparison applies a term-frequency (TF) adjustment on its exact-match level: the non-match probability is replaced by $(u_{\text{exact}}/\max(\text{tf}_l, \text{tf}_r))^w$ where tf is the value's empirical frequency in the reference population, so that a rare email that matches exactly is much stronger evidence than a common one. The lightweight scorer reproduces this adjustment exactly (validated against Splink's own inference to $< 10^{-6}$ on email-exact pairs), so the reported numbers are not distorted by the omission of TF at inference time.

Strict definition of recall. A positive query (identical or close) is counted as a true positive only when Splink matches it to its *own* reference row; matching it to any other row is recorded as a false negative rather than a hit. The evaluation therefore decodes, for each query, the reference-index of the best-matched candidate and compares it to the query's true reference (candidate ids embed the reference position), so recall counts the query that found the correct reference row rather than merely a pair that cleared the threshold. This strict definition is the one used throughout the paper.

These metrics are measured at the *batched* operating point — the lightweight scorer over the whole 40,013-query deck — which is a batch throughput setting, not a per-request estimate: dividing the deck time by the query count gives an amortized ≈ 32.5 ms/query, but that is a throughput bound and is *not* the latency an online call to the resolver experiences. The honest online number is the *cold, per-query* latency of the production path `PersonEntityResolver.resolve`, which runs FAISS blocking and then the lightweight Splink-trained scorer for that query alone. Measured on 30 close-variant queries over the persisted 50,000-record index (artifact `results/erwhitepaper/online_resolver_latency.json`), this was distributed around a median of ≈ 45 ms/query (mean `44.9` ms, p95 `56.0` ms, min `30.4` ms, max `58.1` ms). The artifact also re-

ports each stage timed *independently* (not summed): embedding ≈ 27 ms, FAISS blocking ≈ 29 ms, and the scorer ≈ 15 ms per query. These are separate wall-clock measurements of the same query (so they overlap and do not sum to the ≈ 45 ms end-to-end median); they are reported to show the scorer is a small fraction of the online path, not to decompose the total additively. Because the scorer only evaluates the comparison SQL against a shared connection and never constructs a Splink `Linker`, this cold per-query latency (tens of milliseconds) is an order of magnitude below the previous path that built a fresh Splink `Linker` and ran a separate `predict` per request (which measured a median ≈ 0.43 s/query). All of these are run- and hardware-dependent baselines, not guarantees.

Blocking recall on the same query set. The confusion-matrix run also records whether each positive query's true reference was among the top-k FAISS neighbours. On this exact 40,013-query set the default top-20 blocking recall is 99.61% overall (identical queries 100%, initial-led-name and identity-typo queries 99.7%, close same-entity 97.7%). Of the 600 false negatives, 465 were retrieved but then rejected by the matcher (459 initialled-first-name, 2 identity-typo, 4 close same-entity; the artifact's `blocked_in_but_rejected` field), and only 135 were blocking misses. The binding stage therefore leans on error kind: *blocking* binds for the close-variant positives (a 97.7% blocking ceiling caps linkage recall), while the *matcher* binds for initialled-first-name queries (99.7% blocked but only 90.5% linked). Blocking recall and final linkage recall are measured here from the same artifact and query set, so the two can be compared stage by stage without mixing experiments.

Negative-query hardness. The unrelated queries are generated with an independent Faker seed, and none of the 5,000 negatives shared all three identity fields (first name, last name, date of birth) with its associated reference record in this run (the artifact's `identity_collisions` field records the count). The negatives are therefore separable by identity evidence in principle; genuinely hard negatives — unrelated people who share a full identity signature — would be expected only on larger or more name-dense real populations, and are part of the data-dependence caveats of Section 10.

The confusion-matrix protocol is rerun with a fixed seed (42), so the same queried population and candidate set are regenerated deterministically. The reported counts are nevertheless subject to small run-to-run variation: Splink's inference runs on DuckDB with parallel workers, and repeated runs of `experiment_confusion_matrix.py` under the same settings can differ by a few counts in tens of thousands. Tables should always be read against the persisted artifact named in the caption rather than remembered as exact round numbers, and the conclusions — which rest on metric differences of whole percentage points, not on single-digit counts — are insensitive to this variation.

8.1 Blocking recall under clerical perturbations

Blocking recall is the ceiling of the whole pipeline: the matching stage only ever decides among the k candidates the blocker returns, so a query whose true reference is *not* blocked is a guaranteed false negative regardless of m/u or τ, and a reference that is blocked only at a low rank forces the matcher to disambiguate among near-neighbour look-alikes — exactly the operating region where false positives arise. The confusion-matrix result above quantifies this ceiling on the perturbed deck (top-20 recall 99.61% overall). This subsection measures the same ceiling at both rank 1 and rank 20 — and across many more query types — under the clerical-error perturbations a real deployment must tolerate, perturbing each query with exactly one of the six error types of `person_perturbation.PersonPerturbator`: initialled first name; a typo in first/last name or

date of birth; an address typo; address denormalisation; an email typo; and the outright removal of the address or the email. Typos draw uniformly from additions, deletions, substitutions, and metathesis, and every query is a guaranteed-different duplicate of its base record.

The protocol (`experiment_recall_perturbed.py`) samples 500 base records *per perturbation type* (restricted to records that carry the field the perturbation needs), perturbs each with a seeded perturbator, embeds the query with the same pinned model that built the persisted 50,000-record index (revision from `model_pins.py`), and performs a single top-20 retrieval. Both recall@1 (the base is the best-matching candidate) and recall@20 (the base is inside the candidate set the linker will see) are read from that one retrieval. The experiment is model-repeatable: pointing `-index-dir` at a store built with a different embedding model re-measures the ceiling for that model, exactly the comparison the alternative-embedding research of Section 10 calls for.

Table 8 reports the per-type recall for the MiniLM index under the CPU torch backend (artifact `results/erwhitepaper/recall_perturbed_results.json`). The embedding-model comparison of Table 13 re-runs the identical protocol through the OpenVINO backend, so its MiniLM row differs by a few tenths of a point (e.g. missing-optional recall@1 73.6% here versus 73.4% there); the difference is the inference backend, not the query set. Three observations follow. First, retrieval is essentially immune to single-character address and email typos and to address denormalisation (recall@20 = 100%, recall@1 $\geq$ 99.6%): the mean-pooled row embedding treats "123 Main Street" and "123 Main St" / a redrawn digit as the same neighbourhood. Second, the failures all come from *structural* changes to the text: deleting one whole field (missing optional) costs the most recall@1 (73.6%), initialling the first name costs R@1 the most among the name changes (82.6%), and a typo in identity fields costs 87.6% at rank 1. These are exactly the perturbations a real matching operator cannot avoid, and they matter even though they stay inside the model's own representation. Third, the mean recall@20 of 98.63% here is the same stage-0 ceiling the perturbed confusion-matrix run observes per-kind before linkage (blocking recall 99.61% overall; Table 7): across perturbations, the blocking stage still retrieves the true base almost always, so the linkage stage remains the stage that must not waste the retrieved candidate (> 1% of queries are still lost before the matcher sees them). The gap between recall@1 and recall@20 (90.6% vs. 98.63%) is the part of the ceiling that the matching stage can, in principle, recover: rank information below the top candidate is exactly where τ re-ranking spends its effort.

Table 8: Blocking recall of the persisted 50,000-record index under each of the six `PersonPerturbator` error types (500 queries per type; artifact `results/erwhitepaper/recall_perturbed_results.json`, seed 42). recall@1: the true base is the top returned candidate; recall@20: it is inside the top-20 candidate set the linker sees.

Perturbation	Recall@1	Recall@20	Queries
Initialled first name	82.6%	95.8%	500
Typo in identity (name or DOB)	87.6%	98.0%	500
Typo in address	100.0%	100.0%	500
Address denormalisation	100.0%	100.0%	500
Typo in email	99.6%	100.0%	500
Missing address or email	73.6%	98.0%	500
Mean	90.6%	98.63%	3,000

Also note that the ceiling is *tightened* by the query-generation protocol itself: recall@1 is below recall@20 in every row, so a fixed top-20 candidate budget does a strictly better job of keeping the true base available to the linker than simply returning the nearest vector — this is consistent with

the confusion-matrix finding that the prepared corpus is close to the blocking ceiling, and with the F1-budget argument in Section 10 that the majority of the few residual false negatives originate before linkage, not inside it.

8.2 Trained versus untrained linkage parameters

All figures reported so far use Splink's default, untrained m/u values together with a fixed match prior of 0.0001. Because the whitepaper's recommendations depend on the measured operating point, we quantify whether calibrating m/u changes the results materially. We reuse the 50,000-record reference index and evaluate the same perturbed query deck (identical + six clerical perturbations + close + unrelated per row; 15,996 queries for 2,000 rows) at $\tau = 0.85$ and $k = 20$, holding the match prior at 0.0001 so that the comparison isolates the effect of m/u alone.

Two calibration schemes were considered. The first is Splink's unsupervised expectation maximisation over the reference population. Candidate pairs for EM are generated by exact-equality blocking on two keys, `first_name` and `date_of_birth` (`block_on("first_name")` and `block_on("date_of_birth")`, i.e. the SQL conditions l.first_name = r.first_name and l.date_of_birth = r.date_of_birth), and the same two keys are used to estimate the match prior. Because a de-duplicated reference contains essentially no true duplicate pairs, EM there fits m/u from "different people who happen to share a name or date of birth," which can inflate match probabilities; whether this helps or hurts is data-dependent, as reflected in the benchmark below: on the 100,000-record duplicate-bearing benchmark the EM variant is the *weakest* of the three (F1 94.86%, Table 10), because it over-commits its refitted error terms to the value-equality training collisions; the untrained defaults are the strongest there (F1 97.66%), while on the de-duplicated 50,000-record reference the recommended route is the supervised calibration of the second scheme. The practical caveat stands: unsupervised EM is only as sound as the candidate-pair structure it is fitted over, so its match prior and resulting scores should be verified on a deployment-representative sample rather than trusted by default.

The second, recommended scheme calibrates each comparison level's m and u directly from labelled pairs generated by the same process as the confusion experiment: identical rows and close-variant rows are labelled matches, and independently generated unrelated rows are labelled non-matches. Empirical level probabilities are computed over these pairs with Laplace smoothing (0.5), and null levels are left to Splink's data-driven handling. This supervised calibration is defensible because it uses genuine match and non-match evidence rather than the accidental shared-field structure of the reference population.

Table 9: Trained versus untrained linkage parameters over 50,000 reference records and 15,996 queries ($\tau = 0.85$, $k = 20$, prior 0.0001; artifact `results/erwhitepaper/mu_calibration_results.json`). Evaluation is entity-disjoint: calibration pairs are built from the first 2,000 reference people and evaluation queries from the next 2,000.

Variant	Precision	Recall	F1
Untrained (default m/u)	99.55%	96.59%	98.05%
Trained (supervised, labelled pairs)	99.99%	97.61%	98.79%

Table 9 compares the two variants at the fixed operating point under an entity-disjoint evaluation (the calibration pairs and the evaluated queries come from disjoint reference entities). With the online matcher's weight tables, supervised calibration is well behaved and *improves* the operating

point: on the harder perturbed deck, recall rises from 96.59% to 97.61% and precision from 99.55% to 99.99%, giving F1 of 98.79% versus 98.05% untrained: calibration eliminates the false positives entirely (60 to 0) and cuts the false negatives from 477 to 335, so the worse-but-real per-kind errors of the perturbed deck (the initial-first-name and identity-typo positives) are exactly what the calibrated weights repair. Under this scorer, fitting m/u on labelled pairs is therefore worth doing — it sharpens precision and recall together. How far that advantage generalises depends on the label quality and the operating prior (Section B); the duplicate-bearing benchmark of Table 10 probes the same question with true duplicates present.

These results show that the effect of calibration is *operating-point- and deck-dependent* rather than uniformly monotone. Supervised calibration from labelled pairs improves F1 on the deduplicated 50,000-record reference (98.79% versus 98.05% untrained; Section 8.2, Table 9), but on the duplicate-bearing 100,000-record benchmark the untrained defaults are the strongest variant and the calibrated schemes trade recall for precision (Table 10). The one genuinely sensitivity point, consistent across all variants, is the choice of the match prior: the default configuration is a coherent bundle, so any change to m/u should be accompanied by re-estimation or at least re-verification of the prior and re-selection of τ on a validation set; otherwise metrics are measured at an unintended operating point. Because F1 alone does not separate ranking discrimination from probabilistic calibration, F1 should be read together with calibration and cost metrics on a held-out set.

8.3 Large duplicate-bearing benchmark at 100K rows

The experiments above evaluate a near-duplicate-free reference. To stress the resolver under a realistic mix of true duplicates and unrelated records, `experiment_duplicate_benchmark.py` builds a 100,000-record base population and adds a near-duplicate twin (small field differences at the same 0.15 variation rate) for 3% of base records, yielding a 103,000-record reference index that genuinely contains matching records. The evaluation is entity-disjoint: half of the matched base records contribute labelled duplicate pairs to the supervised calibration, and the other half generate the positive queries, so the trained m/u are never evaluated on entities they were fitted from. The evaluation uses the perturbed deck of Section 8: the 1,500 evaluation bases each emit one positive per applicable clerical perturbation kind plus the close-variant positive (9,045 positive queries across the six kinds) and 1,500 independently generated unrelated negatives, for 10,545 queries total. Queries are blocked to the top 20 FAISS neighbours and scored with Splink at $\tau = 0.85$; only the linkage parameters differ. Three schemes are compared: the untrained default m/u with a fixed prior of 0.0001; supervised calibration of m/u from labelled duplicate/non-match pairs; and unsupervised expectation maximisation over the reference population, which here contains genuine duplicate pairs for EM to exploit. As in the calibration section, the EM candidate pairs are generated by exact-equality blocking on the `first_name` and `date_of_birth` keys. The results are compared in Table 10.

In confusion-matrix terms the untrained variant produced TP = 8,692, FN = 353, FP = 64, TN = 1,436; supervised TP = 8,592, FN = 453, FP = 0, TN = 1,500; and EM TP = 8,160, FN = 885, FP = 0, TN = 1,500. On this harder perturbed deck the ranking reverses relative to the three-way deck: the untrained bundle has the highest recall (96.10%) and the highest F1 (97.66%), because its default weights transfer across the six perturbation kinds; the calibrated schemes eliminate false positives completely (precision 100%, 0 FP) but pay a recall cost (supervised 94.99%, EM 90.22%) large enough that F1 falls below untrained. The per-kind detail in the artifact shows all three schemes recover address/email perturbations and the missing-field variants essentially perfectly, and the whole difference concentrates in the initial-first-name and identity-typo positives

Table 10: Large duplicate-bearing benchmark on the perturbed deck: 100,000 base records plus 3,000 duplicates (103,000-record index), 10,545 entity-disjoint queries ($\tau = 0.85$, $k = 20$; artifact `results/erwhitepaper/training_results.json`).

Variant	Precision	Recall	F1
Untrained (default m/u, prior 0.0001)	99.27%	96.10%	97.66%
Supervised (labelled pairs)	100.00%	94.99%	97.43%
Unsupervised (EM, prior 0.0071)	100.00%	90.22%	94.86%

— the same structural-noise kinds that stress the retrieval stage in Sections 8.1 and 8. EM is hurt most (F1 94.86%), exactly because it re-fits the error terms on value-equality collisions and over-commits them to the training pairs. The practical read-across from Section 8.2 is therefore subtle: supervised calibration reliably raises precision but trades recall on the duplicate-rich population, so which deployment should use depends on whether precision or recall/F1 dominates the cost model. Reported figures must always be tied to the exact deck and parameter configuration used, with a validation-set re-select of the prior and threshold when any component changes.

8.4 Decision-threshold and address-weight sensitivity

Two companions alter the operating point. The *threshold* sweep reuses the 5,000-record / 40,013-query perturbed confusion-matrix population: candidate pairs are emitted at threshold zero once and each decision threshold applied afterwards. The *address-weight* sweep is a separate, reduced-scale run (1,200 records / 3,600 queries; artifact `results/erwhitepaper/f1_address_sweep_results.json`) that scales the address-agreement probabilities by factors 0.6–0.9 against full strength. The two findings below concern τ (where tuning helps, but only marginally) and the address weight (where it does not, on this data set); the address findings are interpreted together with the temporal-decay analysis in Section 3.4.

Raising the decision threshold improves F1 only marginally. At the default (full-strength) address comparison, increasing τ from 0.85 to 0.95 changes F1 imperceptibly (99.104% to 99.116%): false positives fall (22 to 4) but recall already sits below its ceiling here (98.29%), so the higher threshold trades a little recall (98.29% to 98.26%) for a small precision gain (99.94% to 99.99%) rather than buying F1. Table 11 reports the trade-off.

Table 11: F1 versus decision threshold at full address weight on the 5,000-record perturbed confusion-matrix population (40,013 queries; artifact `results/erwhitepaper/f1_sweep_results.json`).

τ	Precision	Recall	F1
0.85	99.94%	98.29%	99.104%
0.87	99.94%	98.26%	99.093%
0.90	99.94%	98.26%	99.093%
0.92	99.94%	98.26%	99.093%
0.95	99.99%	98.26%	99.116%

At $\tau = 0.95$ the matrix is TP = 34,403, FN = 610, FP = 4, TN = 4,996: false positives fall from 22 to 4 (precision 99.94% to 99.99%), but recall falls from 98.29% to 98.26% (F1 99.104% to

99.116%). As in the earlier population, the decision threshold is not a strong lever here, because blocking recall — not the threshold — is the binding constraint: the scorer's own false-positive rate is already tiny (22 in 5,000 negatives), so raising τ mostly trades recall away. Threshold tuning remains worthwhile as a validation-set step, but it buys little on this population.

Weakening the address comparison has no effect. The intuition that address is too volatile to trust as an identity signal was tested by scaling the address-agreement probabilities down ("weakening" the comparison) and re-running the reduced-scale sweep for strengths 0.6, 0.8, 0.9 against full-strength 1.0 (artifact `results/erwhitepaper/f1_address_sweep_results.json`). On that reduced deck the address strength had *no effect*: every strength and threshold produced the same confusion matrix and F1 of 99.64%, because with 30% of addresses missing independently and the surviving synthetic addresses being long and distinctive, the address agreement is far above the decision boundary for the positives and its scaling never flips a decision. Downgrading the address evidence therefore neither helps nor hurts on this data set. The reason, the scope caveats, and how this result sits alongside the temporal-decay evidence are discussed together in Section 3.4.

9 Real-Schema Replication with Synthetic Mutations: NC Voter Data

To test whether the pipeline behaves on real records rather than synthetic ones, we replicated the evaluation on North Carolina voter-registration data [18]. A statewide export was filtered to the fields relevant here — first and last name, birth year (the export carries no full birth date and no email) — yielding about 9.2 million records, from which a deterministic uniform sample of 5,000 was used. Because the public registration file is already heavily de-duplicated and contains no labelled duplicate pairs, authentic duplicate registrations are unavailable. We therefore apply a synthetic mutation model to the real records, analogous to the synthetic Canadian data, to create realistic noisy duplicates: names receive one-character typos, addresses are altered or omitted, and birth years occasionally shift by one. This yields genuine record linkage with noise on a real-world schema: positive queries are mutated duplicates whose clean base record is in the index and must be linked across the noise, and negative queries are mutated versions of held-out records that have no correct match in the index.

Using the same Splink configuration as the synthetic experiments (default m/u, top-20 blocking, $\tau = 0.85$; the email comparison is omitted because the data carries no email), a separate blocking-recall probe (artifact `results/erwhitepaper/ncvoter/results_blocking_recall.json`, a 5,000-record index with 1,000 mutated queries) measured FAISS blocking recall of 82.6%, 85.6%, and 88.0% at $k = 5, 10, 20$. On the resolution deck (a 3,000-record index, 1,500 mutated positives plus 1,500 negatives), mutated-duplicate linkage produced TP = 1,284, FN = 216, FP = 0, TN = 1,500, an F1 of 92.2% (recall 85.6%, precision 100%). The resolution FN count (216 of 1,500) is consistent with the fraction of mutations falling below the top-20 blocking recall, so on this data the blocking stage is a primary constraint. The threshold/address-weight sweep confirmed that the address comparison should stay at full weight on this schema: the full-strength address comparison at $\tau = 0.85$ reached F1 92.2%, while weakening the address comparison to 0.8 fell to 89.4% (precision drops to 90.8%, with 134 false positives).

Because $k = 20$ still missed a nontrivial fraction of mutated candidates, we expanded the blocking size to $k = 50$ and $k = 100$ and measured the linkage outcome and its cost, as summarised in Table 12. Larger k raises end-to-end recall from 85.6% ($k = 20$) to 87.5% ($k = 50$) and 88.4% ($k = 100$). The gains are monotonic but strongly diminishing (only +28 and then +14 true

positives), while precision stays ≈ 100% (exactly one false positive at $k = 100$), confirming the linkage model is robust to candidate inflation on this schema. The recall gains come from the larger candidate set: as k grows, more mutated duplicates of previously missed bases enter the top-k and are matched, so end-to-end recall rises even though the scorer adds no false positives at $k \leq 50$ (precision 100%; exactly one false positive at $k = 100$). The effective recall ceiling is still set by the retrieval stage, since the 3,000-record resolution deck's own blocking recall at $k = 20$ was 88.0% (measured on the separate 5,000-record blocking probe). The cost of k on the linkage scorer is small: scoring the whole 3,000-query deck with the lightweight trained scorer took 23.9 s at $k = 20$, 25.3 s at $k = 50$, and 26.2 s at $k = 100$ — essentially flat across the fivefold candidate growth (a 1.10× rise, ≈ 8.0 ms/query amortised), because this scorer's per-query overhead dominates at these low candidate counts. These are batched-deck throughput numbers, not the per-query online latency of Section 8; FAISS blocking time stayed effectively constant across k.

Table 12: Effect of blocking size on mutated NC-voter linkage (1,500 positives, 1,500 negatives = 3,000 queries, $\tau = 0.85$; artifacts `results/erwhitepaper/ncvoter/results_resolution.json` and the k=50, k=100 variants; each uses a 3,000-record index). Larger k recovers more mutated duplicates, but gains are diminishing. "Scorer time" is the wall-clock of the lightweight trained scorer (whitepaper Section 10.7 scoring engine) over the *entire* 3,000-query deck in one batched pass (≈ 8.0 ms/query amortised), not the per-query online latency of Section 8; it is essentially flat as k grows because the scorer's per-query overhead dominates at these candidate counts. Blocking time stays effectively constant.

k	Blocking recall	End-to-end recall	Precision	F1	Scorer time (4,500 queries)
20	88.0%	85.6%	100.0%	92.2%	23.9 s
50	90.9%	87.5%	100.0%	93.3%	25.3 s
100	92.6%	88.4%	99.9%	93.8%	26.2 s

These figures are more informative than exact self-resolution but remain bounded by the source. First, the registration file is still well de-duplicated, so every positive is a synthetic mutation of an otherwise distinct base record rather than a genuine duplicate registration; the mutation model proxies noisy duplicate intake (typos, address churn, missing fields) but cannot reproduce real voter turnover or duplicate-file scenarios. Second, the export contains only a birth year and no email, so the date-of-birth and email comparisons — two identity signals in the synthetic experiments — are only weakly tested: date-of-birth errors are not properly exercised, and email is absent entirely. The replication therefore confirms the pipeline on a real schema with injected noise while leaving authentic duplicate, full-DOB, and email scenarios to the data sets discussed in Section 10.

10 Further Research

10.1 Alternative embedding engines

The current MiniLM encoder is a useful baseline, but it is not specialized for entity resolution or structured rows. Candidate alternatives include:

- larger sentence-transformer models for stronger semantic representations;
- multilingual encoders for names and addresses across language communities;
- character n-gram or TF–IDF vectors for typo-sensitive retrieval;

- domain-adapted contrastive encoders trained on positive and hard-negative person pairs;
- graph-based universal tabular embeddings such as those proposed by Franz et al. [7]; and
- hybrid retrieval that combines dense similarity with lexical or field-specific indexes.

The comparison should measure not only final linkage quality but also whether the embedding preserves the true match in the top 20. A particularly promising approach is multi-view retrieval: learn separate views for identity fields, contact fields, and address fields, then combine their scores before Splink. This could reduce the tendency of a long address to dominate a short but highly identifying date of birth.

Measured effect of the embedding model on blocking recall. To make the "other models" discussion concrete, we ran the clerical-perturbation blocking-recall protocol of Section 8.1 — the same 50,000-record base population, the same 500 seeded queries per perturbation type, and the same single top-20 retrieval — for each candidate embedding (`experiment_recall_perturbed_models.py`), so the only thing that changes between rows is the embedder. Every model is revision-pinned (the exact Hugging Face commit is recorded in the artifact `results/erwhitepaper/recall_perturbed_models_result` all-MiniLM-L6-v2 at `1110a243fdf4`, MongoDB's mdbr-leaf-mt at `1ed41b22ce16`, infgrad's stella-base-en-v2 at `c9e80ff9892d`, and avsolatorio's GIST-Embedding-v0 at `bf6b2e55e92f`. Inference ran through the OpenVINO backend on the Intel Iris Xe integrated GPU, so the run is CPU/GPU portable and reproducible. Table 13 reports, per model, the embedding dimension and the mean recall@1 / recall@20 over the six perturbation kinds, plus recall@1 for the three perturbation kinds that stressed the MiniLM baseline most (initialled first name, identity typo, missing optional field).

Table 13: Blocking recall under clerical perturbations for four sentence-transformer models on the same 50,000-record base (500 queries per perturbation type, single top-20 retrieval; artifact `results/erwhitepaper/recall_perturbed_models_results.json`). Mean recall@1 and recall@20 average the six perturbation types. Address-level perturbations (address typo, address denormalisation, email typo) recover perfectly everywhere; the spread below is concentrated in name-level noise and missing fields.

Model (revision)	Dim	Mean R@1	Mean R@20	Initial	Identity	Missing opt.
all-MiniLM-L6-v2 (`1110a243`)	384	90.5%	98.6%	82.6%	87.6%	73.4%
mdbr-leaf-mt (`1ed41b22`)	1024	97.1%	99.9%	96.0%	96.6%	90.0%
stella-base-en-v2 (`c9e80ff9`)	768	97.4%	99.9%	96.4%	97.6%	90.6%
GIST-Embedding-v0 (`bf6b2e55`)	768	98.1%	99.9%	98.8%	98.0%	92.0%

Three observations follow. First, the larger and more modern embedders are considerably more robust at *rank 1*: mean recall@1 rises from 90.5% for MiniLM to 97.1–98.1% for mdbr-leaf-mt, stella-base-en-v2, and GIST-Embedding-v0 — a 7–8 point reduction in the top-candidate mistakes the linkage stage has to repair. The gain concentrates exactly where the confusion-matrix protocol does not stress the pipeline: on the structural noise types (initialled first name, name typos, missing optional field), where MiniLM's mean-pooled representation loses a quarter or more of the true bases at rank 1 and the alternatives recover most of them. Second, address-typography noise (address typo, address denormalisation, email typo) is recovered perfectly (100%) by MiniLM and by every alternative, so the differentiation is specifically about identity-level noise and field deletion, not address formatting. Third, at recall@20 the models are almost indistinguishable (98.6% MiniLM versus 99.9% for the others), i.e. the candidate set the linker actually sees differs only slightly; the

observable, operationally relevant difference is at the top of the ranking. These numbers give the headroom a model swap would buy on the confusion-matrix population itself: replacing the default MiniLM encoder with a 768- or 1024-dimension alternative would reduce the rank-1 failures of the blocking stage by about 7–8 points, lowering the false negatives presented to the matcher at τ, before any multi-view or hybrid (dense + lexical) retrieval is even introduced. This is a direct confirmation that embedding choice is a first-class retrieval lever for this workload, and it upgrades the alternatives above from "promising" to "measured".

Relative cost of the alternatives. The recall gains above are not free, and the per-model logs in the same artifact make the trade-off explicit. Index construction over the 50,000-record base on the OpenVINO GPU (one pass over all records) took 82.2 s for all-MiniLM-L6-v2 and 87.4 s for mdbr-leaf-mt, but 435.1 s for stella-base-en-v2 and 430.3 s for GIST-Embedding-v0 — roughly a 5× build-time premium for the two larger models, driven mostly by first-use OpenVINO export for models without a prebuilt IR and by their bigger encoder stacks. Query-side (3,000 perturbed queries, batch-embedded per perturbation type) the same split recurs: MiniLM 9.7 s and mdbr 10.1 s versus stella 35.3 s and GIST 33.7 s, i.e. the per-query embed is roughly 3.5× slower for the large models even on the GPU. mdbr-leaf-mt is therefore the standout on cost-adjusted recall: it matches the small-model build and query cost (within 6% of MiniLM on both) while delivering mean recall@1 of 97.1% — essentially all of the large-model recall gain at a third of their cost. The two bert-large-size alternatives reclaim only the last ≈ 1 point of rank-1 recall above mdbr (97.4–98.1% versus 97.1%), which most deployments would not justify at a 4–5× build and 3.5× query premium; the numbers still argue for re-benchmarking whichever model is operationally cheap on a target GPU, since the ranking of build cost tracks encoder depth, not recall.

These directions are worth also tempering against the measured F1 budget. Measured on the same query set as the headline confusion matrix (Section 8), the default top-20 blocking recall is 99.61% overall (identical 100%, close same-entity 97.7%), and measured end-to-end recall is a close 98.29%: of the 600 false negatives, 465 were retrieved-then-rejected by the matcher (mostly initialled-first-name positives) and 135 were blocking misses, so the binding stage depends on error kind — the *blocker* caps the close-variant positives (blocking recall 97.7%) while the *matcher* fails most of the initialled-first-name retrieved candidates (99.7% blocked, 90.5% linked). The same ceiling holds when the queries are clerical perturbations measured at the blocking stage alone (Section 8.1): mean top-20 blocking recall under the six perturbation types on the 50k index is 98.63% (rank-1 mean 90.6%). A perfect blocker at rank 20 would add at most 0.43 points of recall (99.61% to 100%); the larger, structural gap is at rank 1 — 90.6% mean recall@1 across the six perturbations versus the 98.29% the linker achieves end-to-end on its candidate set — which is where a stronger embedder or multi-view retrieval has visible headroom, particularly for the initial-first-name and close-variation positives. Accordingly, the earlier prioritisation is reinforced: retrieval-side research is currently the higher-impact lever, with the decision threshold a secondary knob as shown by the flat τ sweep above.

10.2 Replication on a real schema with synthetic mutations

The central experiments in this paper are measured on synthetically generated Canadian person records (Faker with a custom Canadian address provider). Synthetic data is valuable because it supplies controlled missingness, exact ground truth, and repeatable duplicate rates, but it embeds assumptions about name distributions, address formats and volatility, email conventions, and how errors arise between records of the same entity. The replication on the North Carolina voter data (Section 9) is a first step toward real data — a real-schema replication in which the positive

pairs are synthetic mutations of held-out real records rather than authentic duplicate registrations. As noted there, it is a near-best-case scenario: it validates the pipeline against real name and address distributions but does not fully exercise cross-record duplicates, naming variation, or birth-date errors. Whether the findings hold more broadly with real data is therefore still an open empirical question, and the experiments should be replicated on non-synthetic, labelled record-linkage data. The replication should run the identical pipeline and protocols — same top-k FAISS blocking, same comparisons and decision machinery, the confusion-matrix three-query protocol, and the trained/untrained/supervised m/u, threshold, and address-weight sweeps — and compare the resulting confusion matrices, F1, and the conclusions drawn from them.

The natural test beds are public benchmark record-linkage data sets with known ground truth. These include the Cora citation data [10], the FEBRL restaurant data [23], and the product and citation pairs from the Abt–Buy, Amazon–Google, and DBLP–ACM benchmarks used in the deep-matcher literature [13] (data sets: https://dbs.uni-leipzig.de/research/projects/benchmark-datasets-for-entity-resolution/). Where a data set lacks the fields used here (date of birth, address, email), the comparison should be restricted to the fields it does provide, or the pipeline should be adapted to the available schema while keeping the methodology fixed. While the benchmark data sets above provide authentic duplicate pairs with ground truth, public registration data such as the North Carolina voter file is a useful but limited test bed: it is heavily de-duplicated by the source and typically lacks a full date of birth, so it contains few authentic duplicate registrations and cannot exercise birth-date errors. The mutation-model replication in Section 9 supplies a noisy-duplicate proxy on a real schema, but authentic duplicates and full-DOB comparisons still require the labelled benchmark or corporate data discussed below.

Such replication serves several specific purposes. First, it tests whether the address field is as informative as the synthetic data suggests: public and production datasets expose the real address volatility discussed in Section 3.4 (generic or unit-only addresses, frequent relocations), so the synthetic finding that weakening the address comparison did not help — and the operating question of when address evidence should be time-decayed — may not transfer to these regimes. Second, real data can reveal whether retraining m/u on the collection itself behaves like the lab setting, where the untrained default outperformed the trained variants, or whether genuinely noisy real duplicates make calibration beneficial. Third, the recall distributions and blocking ceiling measured at $k = 20$ in Section 10 may differ when names collide at realistic rates, changing where the F1 budget lies between the blocking and linkage stages. Finally, replication must handle missing values, inconsistent postal/date formats, and multilingual names, which the synthetic generator controls but real data exposes. Privacy considerations apply because person records are personally identifying; labels and ground truth should be obtained from datasets that are published for research or from de-identified data with controlled access. The goal is not to certify a specific score but to confirm which qualitative conclusions — which levers move F1 and which do not — survive contact with real data.

Beyond public benchmarks, real-world performance is best assessed on corporate data: production master records that an organisation genuinely needs to de-duplicate or match, such as customer, patient, employee, or product records, ideally with a human-validated sample of true duplicates. Public data sets are usually small, masked, or already de-duplicated, whereas corporate data carries the actual error profile the deployment will face — real naming conventions, address churn, partial or corrupted fields, and the true duplicate rate of the workload. That allows the pipeline to be calibrated and its F1 reported against the workload it will actually serve, which is the strongest test of real-world performance. The constraints are access and privacy: matching personnel must have legitimate authority, data transfers need privacy controls, ground-truth labelling should be reviewed for accuracy, and only the minimum necessary evidence should be logged. A

mid-size corporate master (tens of thousands to a few million records) is typically large enough to expose distributional differences that the small synthetic and public sets miss while remaining small enough to run the experiments directly. Such partnerships make it possible to validate both the quantitative results and the qualitative conclusions about which levers, such as the decision threshold, matter most in practice.

10.3 Continuous matcher scores and the Fellegi–Sunter weights

The Splink comparisons in this paper produce *discrete* levels, each with its own m/u probabilities and therefore its own match weight. Learned matchers, by contrast, return a *continuous* score — a deep-network probability (DeepMatcher, Ditto) or an embedding cosine – and the question is how to apply Fellegi–Sunter m/u parameters to such scores rather than to categorical levels.

The standard recipe treats a continuous score x as a comparison level of infinite resolution and forms the likelihood ratio

$$\frac{m(x)}{u(x)} = \frac{f(x \mid M)}{f(x \mid U)},$$

which feeds the same Fellegi–Sunter log-odds as a discrete level weight. The two densities must be estimated from data, by fitting a normal-mixture model to the score distribution, by calibration against labelled pairs, or by EM — a standard Fellegi–Sunter estimation problem. Winkler [21] reviews how comparators' scores are mapped to weights, including the mixture-model estimation of m/u that he and Jaro introduced; Christen's survey [23] describes these techniques only via those primary sources.

The modern, directly relevant line applies the same idea to learned matchers: the end-to-end survey of Christophides et al. [14] positions learned matchers and probabilistic (Fellegi–Sunter) linkage as stages of one entity-resolution pipeline, alongside its blocking stage, and Sadinle and Fienberg [15] generalise the Fellegi–Sunter construction to the multiple-record setting, fitting match patterns from the comparison data by mixture maximum likelihood. Together these yield the practical recipe for this paper's setting: treat the matcher score as the comparison, estimate $f(x \mid M)$ and $f(x \mid U)$ (by mixture modelling as in [21], by calibration against labelled pairs, or by EM; cf. the survey [23]), form $w(x) = \log_2[f(x \mid M)/f(x \mid U)]$, and add it to the FS log-odds exactly like a level weight, with the match prior λ unchanged.

This matters for two reasons in the present work. First, the deep-matcher literature — DeepMatcher [13], DeepER [5], Ditto [16], and large-language-model matchers [17] — all operate on the matching step, deciding match/no-match for the record pairs supplied to them; none addresses the retrieval-side question of whether a far-apart duplicate is a candidate at all. That is the gap the gap-weighted, capture-date-conditional fusion of this paper targets, and it is orthogonal to how the matcher's score is converted into a weight. Second, the m/u calibration caveats already established here apply a fortiori to continuous scores: a score distribution fitted on resemblance-biased candidate pairs (as from dense-neighbour blocking) will inherit the same prior drift as the value-blocking case, so the calibration lesson — anchor the prior and re-tune τ — carries over unchanged.

10.4 Canopy-based parameter training

The Splink parameters trained in this paper — whether by unsupervised expectation maximisation over the reference population (Section 8.2) or by supervised calibration over labelled pairs — are fitted on candidate pairs generated by exact-equality blocking on the `first_name` and `date_of_birth` keys. Embedding blocking offers an alternative candidate-generation strategy for training: instead of value-based equality, candidate pairs could be produced by dense-neighbour clustering of the

same row embeddings used for online blocking. The classic recipe is *canopy clustering*, in which a cheap distance measure places each record into overlapping candidates using two thresholds (a wide capture radius and a tighter one), after which an expensive similarity is applied only within the tighter canopies [10]. Dense top-k retrieval is a direct analogue: each query's retrieved neighbourhood is a canopy, so the online blocker in this paper is already canopy-shaped, and the same construction could supply training pairs.

The argument for doing so is that canopy/embedding candidates are density-derived rather than attribute-derived, so an EM fit over them resembles the collision structure of a real retrieval workload more closely than value-equality blocks do. This direction is well supported in the entity-resolution literature: embedding models have been used to build the candidate sets on which later matchers and training pipelines operate [5], similarity-based (including vector) blocking has been evaluated directly against exact-match blocking [11], pre-trained-embedding blocking variants have been benchmarked head-to-head against each other and against a vector-based state of the art [4], and learned blocking functions selected from labelled seeds can be viewed as data-driven analogues of canopy selection [12]. In all of these the embedding blocker feeds a downstream matcher, and the same feed-forward relationship applies to training the matcher's own parameters.

This is therefore a concrete next step: replace the value-equality training blocks with a `block_id` derived from FAISS clusters (the store already exposes the vector space used for online blocking), and re-fit the Splink m/u and prior on those candidate pairs. This canopy-style two-phase design, and its reuse of the same index for batch deduplication and insertion-time duplicate prevention, is developed experimentally in a companion paper on batch deduplication with a shared vector canopy index [26]. The approach is not yet implemented or measured in this paper, so it is flagged as future work rather than a reported result. Because candidate pairs produced by embedding neighbourhoods are even more resemblance-biased than value-equality collisions, the match prior fitted over them is expected to drift upward relative to a value-blocking fit, so any such prior must be re-anchored and τ re-tuned jointly (Section B).

10.5 Calibration and deployment

Future work should estimate Splink parameters from labelled operational samples, quantify probability calibration, and introduce drift monitoring for names, domains, postal formats, and address conventions (including the temporal decay of address evidence discussed in Section 3.4). Privacy-preserving evaluation is also important: synthetic data is useful for development, but production validation must control access to personally identifying information and log only the minimum necessary evidence.

10.6 Parsing and segmentation

The pipeline currently consumes already-structured fields (the synthetic generator and the NC-voter export supply separate name and address columns), but real-world ingestion is frequently unfielded raw strings. Research should quantify the impact of name and address parsing and segmentation — turning a raw name or block address into first/middle/last/suffix and street/unit/city/state/postcode components and normalizing them — on both the accuracy and the resource usage of the pipeline. On the accuracy axis, the questions are (i) how segmentation errors propagate through the row embedding into blocking recall and then into linkage F1 and calibration: a mis-segmented name or a wrongly split address degrades both the embedding and the string comparisons Splink relies on, and it changes the non-match agreement rate (u) that the linkage observes; and (ii) how the choice of parser — regex heuristics, a statistical parser such as

libpostal, a neural token tagger, or an instruction-tuned LLM — changes those outcomes, measured against deliberately unfielded or segmentation-noisy inputs. On the resource axis, the research should cover parsing cost per record (CPU/GPU time, memory, and per-query latency) and, in particular, where a staged strategy lands in the pipeline's throughput budget when parsed fields are embedded and matched in batch mode — the amortized batched figure, not the per-request online path: a fast deterministic/library tier for the bulk of well-formed records, a lightweight transformer tagger for names, and an LLM reserved for the messy residual tail, whose latency and cost then trade against downstream blocking and linkage. Because segmentation sits upstream of embedding, blocking, and linkage, its errors amortize across every later stage; quantifying that amplification, and the accuracy-versus-resource frontier across parser tiers, is a concrete high-value next step.

10.7 A purpose-built online Fellegi–Sunter matcher

Splink is a batch linkage engine: it scores all blocking pairs in one materialised pass and is not optimised for the single-query, single-block case. The original resolver rebuilt a Splink `Linker` and re-*materialised* Splink's full DuckDB comparison pipeline for every query, which the timing experiment measured at ≈ 0.4–1.9 s/query depending on load (Section 8). The current implementation removes that from the online path: instead of running Splink per query, the pipeline trains m/u with Splink once and scores each query with a purpose-built Fellegi–Sunter matcher (§3.5). This matcher holds the comparison-level weight tables in memory, assigns each candidate the level whose Splink-generated `sql_condition` it satisfies, and computes the thresholded posterior directly — no `Linker` and no DuckDB pipeline is constructed per request. Because the candidate count is bounded by the fixed k (20 in the experiments), the per-query comparison work is a small constant number of field-wise score evaluations over the top-k FAISS candidates, giving a measured cold median latency of ≈ 45 ms/query (embedding ≈ 27 ms, FAISS ≈ 29 ms, scorer ≈ 15 ms) on the 50,000-record index — an order of magnitude faster than the prior per-query Splink path.

Two properties of this matcher are worth making explicit. First, it reproduces Splink's linking decisions on the same fixed candidate sets: the comparison `sql_condition` strings are taken from the very Splink objects `predict` lowers to SQL, so the value-to-level mapping is identical by construction, and the scorer matches Splink's posterior to $\approx 10^{-7}$ mean absolute difference over real candidate pairs, with no classification-level divergence in the confusion-matrix run. Because the scorer no longer carries Splink's term-frequency exact-match adjustment on the email comparison verbatim, a tiny fraction of pairs at probability saturation > 0.999 differ only in the last significant digits of a near-1 probability. Second, it makes the trained m/u directly reusable as plain persisted numbers, so calibration is a matter of writing an updated weights file rather than re-serialising a `Linker`. The remaining, now fully separable work is calibrating those m/u on operationally representative labelled pairs and re-anchoring the prior and threshold jointly (Sections 8.2 and B); the fast online matcher is the vehicle through which any calibrated weights are served, so the honest online latency to report is now the cold resolver number of ≈ 45 ms, not the amortized batch figure.

11 Conclusion

The implemented architecture is an *incremental*, online resolver: it combines a fast vector retrieval layer with an interpretable probabilistic linkage layer and serves each query independently against a fixed reference population. It is sized and validated for per-request entity resolution (API and streaming-insert workloads); batch linkage of two large stored populations is outside its design

and is better served by a dedicated batch linker that rebuilds and clusters the full pairwise comparison graph. FAISS narrows a 50,000-record population to 20 candidates, while a purpose-built Fellegi–Sunter matcher (trained with Splink, served by a lightweight scorer, §10.7) compares identity, contact, and address fields under a configurable decision threshold without per-query linker construction. Persisting the index and metadata makes the workflow operationally reusable. The most important next steps are to train and calibrate the m/u parameters on labelled pairs and jointly re-anchor the prior and threshold, benchmark alternative row and tabular embedding engines using blocking recall as a first-class metric, and — because the fast online matcher is already in place — serve any calibrated weights through it directly.

The measured experiments also isolate which control levers move F1 across the validation population and which do not. On the perturbed deck the binding stage depends on error kind: *blocking* limits the close-variant positives (top-20 blocking recall 97.7% on the confusion-matrix population) while the *matcher* fails most of the retrieved initialled-first-name positives, and the rank-1 gap (90.6% mean recall@1 under the six perturbations vs. the 98.29% the linkage stage achieves end-to-end) is the headroom a stronger embedder or multi-view retrieval could recover. Raising τ from 0.85 to 0.95 buys little (F1 99.104% to 99.116%) because recall, not false positives, is scarce. The row serialization strategy shows a small but repeatable advantage: compact raised top-20 blocking recall to 99.76% and F1 to 99.175% in the single-seed Section 7 run (99.175% at τ=0.85 versus 99.104% default), with a five-seed aggregation confirming a small but outside-seed-noise compact F1 edge (+0.057 points, Section 7). Weakening the address comparison (scaling agreement probabilities down by factors 0.6–0.9) had no effect on the reduced-scale address sweep (F1 99.64% at every strength; artifact `f1_address_sweep_results.json`). Supervised calibration from labelled pairs improves F1 at the inherited operating point (98.79% versus 98.05% on the 50 k index). The effect is deck-dependent, however: on the duplicate-bearing 100 k benchmark the untrained defaults are the strongest variant (F1 97.66%, versus 97.43% supervised and 94.86% EM), because the calibrated schemes trade recall for precision (Section 8.2). The remaining, controllable sensitivity is the choice of the match prior, which should be estimated or verified rather than left to an unsupervised fit. Operationally representative labels and joint threshold/prior calibration therefore remain necessary to lock in the supervised gain at deployment, and the fast online matcher is the vehicle through which any calibrated weights are served.

A Deriving the Decision Threshold τ

The decision threshold τ should be treated as a statistical boundary rather than an arbitrary heuristic. The standard algorithms for deriving it are grouped here by whether labelled ground truth is available.

A.1 With labelled ground truth (supervised derivation)

If a labelled validation set of true matches and non-matches exists, classical optimisation applies.

Expected-cost minimisation. If a false positive carries a business cost C_{FP} and a false negative a cost C_{FN}, then, assuming correct decisions carry no cost and the posterior $P(M \mid \gamma)$ is well calibrated, decision theory states that the expected loss is minimised by declaring a pair a match exactly when its posterior match probability satisfies $P(M \mid \gamma) \geq \tau^*$, with

$$\tau^* = \frac{C_{FP}}{C_{FP} + C_{FN}} .$$

For example, if merging two distinct accounts (C_{FP}) is twenty times more costly than failing to link two records of the same person (C_{FN}), then $\tau^* = 20/(20+1) \approx 0.95$. This is the standard Bayes decision-theoretic threshold; the optimality criterion in Fellegi–Sunter [1, 19], by contrast, is error-bound based: for fixed upper bounds on the false-match and false-nonmatch error rates, minimise the expected number of pairs left undecided, yielding the two-threshold (link / possible-link / non-link) rule rather than a single cost ratio. Two caveats apply to the cost formula: it presumes costs that are constant across pairs and zero cost for correct decisions (if a correct merge itself has an operational cost, or costs vary by pair, the threshold shifts and is no longer a single global value); and it is optimal per pair only when pair decisions are independent — under transitivity and one-to-one constraints it is a heuristic operating point (see the transitivity rule below). With labels in hand, a more robust alternative is to sweep τ on the validation set to minimise empirical cost directly, which does not depend on calibration.

F1 maximisation. Alternatively one maximises the empirical F1 score by sweeping candidate thresholds:

$$\tau^* = \underset{\tau \in [0.5, 0.999]}{\arg\max} \frac{2 \cdot \text{Precision}(\tau) \cdot \text{Recall}(\tau)}{\text{Precision}(\tau) + \text{Recall}(\tau)}.$$

Precision-recall trade-offs and threshold selection are standard practice for skewed classification [22, 23]. The threshold and address-weight sweeps in this paper (Table 11 and the NC-voter experiments) instantiate this supervised rule on a labelled validation set.

A.2 Without labelled ground truth (unsupervised derivation)

When labels are absent, empirical precision and recall cannot be measured directly, and the boundary must be inferred from the data.

Expected-F1 maximisation. If the match probabilities $p_j = P(M \mid \gamma_j)$ estimated by the fitted model (e.g., by expectation maximisation) are well calibrated, the expected counts at any threshold τ can be computed without labels. With $D_\tau = \{j : p_j \geq \tau\}$ the pairs classified as matches, the expected true positives, false positives, and false negatives are

$$E[TP(\tau)] = \sum_{j \in D_\tau} p_j, \qquad E[FP(\tau)] = \sum_{j \in D_\tau} (1 - p_j), \qquad E[FN(\tau)] = \sum_{j \notin D_\tau} p_j,$$

so the plug-in F1 computed from these expected counts (an approximation valid for large $|D_\tau|$ via the delta method, and treating pairs as independent Bernoulli draws) is

$$E[F_1(\tau)] = \frac{2 \sum_{j \in D_\tau} p_j}{|D_\tau| + \sum_j p_j},$$

where $\sum_j p_j$ is the total expected number of matches in the candidate-pair set S. One then selects $\tau^* = \arg\max_\tau E[F_1(\tau)]$ over a dense grid. This is a direct use of the calibrated posterior probabilities produced by the mixture/EM fitting of the Fellegi–Sunter model as reviewed in [21]; it approximates the cost-based rule when the two error types are weighted equally.

Gaussian-mixture valley detection. The Fellegi–Sunter weights $w_j = \log_2 R(\gamma_j)$ are typically bimodal: a large peak of low-weight non-matches and a smaller high-weight match peak. Fitting a two-component Gaussian mixture, in the weight-fitting tradition of [21], to the weights yields

densities $f_U(w)$ and $f_M(w)$ for the non-match and match components; the threshold is placed at the intersection

$$p\, f_M(w^*) = (1-p)\, f_U(w^*),$$

with p the prior match probability, and the weight w^* converted back to a probability via the Bayes relation $\tau^* = \left(1 + \frac{1-p}{p}\, 2^{-w^*}\right)^{-1}$. This places the boundary at the statistical "valley" separating the two populations; frequency-based and decision-rule refinements of the Fellegi–Sunter weights also assume such well-separated weight distributions [20].

Graph-transitivity consistency. Linkage is expected to be transitive (if A matches B and B matches C, then A matches C). For a candidate τ, build a graph with an edge between records i, j when $p_{ij} \geq \tau$, count transitivity violations (e.g., A connected to B and B to C but A not to C), and select the τ that minimises violations while maximising cluster cohesion, for example the silhouette score of the resulting connected components. Record-linkage clustering under transitive closure is standard in the field [23].

B Tuning the Match Prior λ

The match prior $\lambda =$ *probability_two_random_records_match* and the decision threshold τ jointly determine the operating point, and the supervised/EM calibration results of Section 8.2 were measured at a fixed prior. Because different deployments have different base match rates and error costs, this appendix outlines an algorithm that tunes the prior and the decision threshold together, keeping them a coherent pair. The default value $\lambda_0 = 0.0001$ is the Splink default; the recipe below starts there and re-anchors it to a defensible estimate when one is available.

Step 0 (baseline). Set $\lambda_0 = 0.0001$ (the Splink default).

Step 1 (estimate a principled prior; do not use EM's free prior).

- *With labels:* $\lambda_{\text{est}} = \frac{\text{proportion of candidate pairs that are true matches}}{\text{blocking recall}}$, i.e. the base match rate in random-record-pair space rather than within the blocked candidate set.
- *Without labels:* use Splink's estimator `estimate_probability_two_random_records_match`, called with the blocking rules and a `recall`; it divides the expected matches by the blocking coverage. If no defensible match-rate assumption exists, keep $\lambda_0 = 0.0001$.

Step 2 (fit m/u with the prior pinned). Run Splink EM with `fix_probability_two_random_records_match = True` so that EM estimates m/u only, or keep the untrained defaults. This keeps λ anchored while the evidence weights may change, so that the fitted m/u and the prior stay a coherent pair.

Step 3 (choose (λ, τ) jointly on validation). Grid over $\lambda \in \{\lambda_0, \lambda_{\text{est}}\} \times 10^{-k}$ and $\tau \in [0.5, 0.999]$; for each cell score all pairs and evaluate the objective. *With labels:* decision cost $C_{FP} \cdot FP + C_{FN} \cdot FN$ (or F1), selecting the minimiser/argmax. *Without labels:* the expected-F1 or expected-cost from the calibrated posteriors, or the GMM-valley threshold, as defined in Appendix A.

Step 4 (coherence guard). Verify that the chosen operating point is internally consistent: the mean posterior probability of non-match pairs should stay low (so that λ is not inflating match scores), and false positives and false negatives should move monotonically with τ around the chosen λ. This guard keeps the prior from silently shifting the operating point when m/u are retrained.

Step 5 (freeze and report). Lock (λ^*, τ^*) on validation and measure once on a held-out test set, reporting λ, τ, F1, and the false-positive/false-negative/cost outcome.

Why this recipe: (i) λ is anchored — the default or a defensible estimate — instead of being re-estimated freely by an unsupervised fit, so match scores are not shifted by an inflated prior; (ii) λ and τ are tuned together, matching the coherent-bundle rule that no single element is changed in isolation; (iii) the coherence guard distinguishes an over-inflated prior from a genuine operating-point choice before the numbers are trusted. Scope note: the blocking-adjusted prior estimate is Splink-native; the joint grid and guard are the contribution here.

Use of Artificial Intelligence

During the preparation of this work the author used an AI coding and writing assistant — the DeepSeek V4 Flash 0731 large language model accessed through an interactive coding agent — to assist with drafting and editing the manuscript, authoring and debugging the code that implements and evaluates the pipeline, and running the experiments and automation that produced the reported numbers. The tool was not listed as an author, as it cannot take responsibility for the work. All numerical results reported in this paper were obtained by executing the repository code and were independently re-checked for accuracy. Because generative assistants can hallucinate citations, every bibliographic entry in the reference list was re-verified by the author against the publisher, Crossref, DBLP, or arXiv records in August 2026; the record for each entry states its DOI or stable archive identifier, and any entry that could not be verified was removed or replaced. Every claim, table, figure, result, and conclusion was thoroughly reviewed and verified by the author, who takes full responsibility for the content of this document, including any remaining errors and/or omissions.

References

[1] Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. Journal of the American Statistical Association, 64(328):1183–1210, 1969. https://doi.org/10.1080/01621459.1969.10501049.

[2] Robin Linacre, Sam Lindsay, Theodore Manassis, Zoe Slade, Tom Hepworth, Ross Kennedy, and Andrew Bond. Splink: Free software for probabilistic record linkage at scale. International Journal of Population Data Science, 7(3), 2022. https://doi.org/10.23889/ijpds.v7i3.1794. Software repository: https://github.com/moj-analytical-services/splink.

[3] Tianshu Wang, Hongyu Lin, Xianpei Han, Xiaoyang Chen, Boxi Cao, and Le Sun. Towards universal dense blocking for entity resolution. arXiv preprint arXiv:2404.14831, 2024. https://arxiv.org/abs/2404.14831. Code repository: https://github.com/tshu-w/uniblocker.

[4] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, and Manolis Koubarakis. Pre-trained embeddings for entity resolution: An experimental analysis. Proceedings of the VLDB Endowment, 16(9):2225–2238, 2023. https://doi.org/10.14778/3598581.3598594.

[5] Muhammad Ebraheem, Saravanan Thirumuruganathan, Shafiq Joty, Mourad Ouzzani, and Nan Tang. Distributed representations of tuples for entity resolution. Proceedings of the VLDB Endowment, 11(11):1454–1467, 2018. https://doi.org/10.14778/3236187.3236198.

[6] Forest Gregg and Derek Eder. Dedupe: A Python library for accurate and scalable fuzzy matching, record deduplication and entity resolution (version 2.x). https://github.com/dedupeio/dedupe.

[7] Astrid Franz, Frederik Hoppe, Marianne Michaelis, and Udo Göbel. Universal embeddings of tabular data. arXiv:2507.05904v1, 2025. https://arxiv.org/abs/2507.05904v1.

[8] Liane Vogel, Kavitha Srinivas, Niharika D'Souza, Sola Shirai, Oktie Hassanzadeh, and Horst Samulowitz. Towards universal tabular embeddings: A benchmark across data tasks. arXiv:2604.21696v1, 2026. https://arxiv.org/abs/2604.21696v1.

[9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. IEEE Transactions on Big Data, 7(3):535–547, 2021. https://doi.org/10.1109/TBDATA.2019.2921572. Preprint: https://arxiv.org/abs/1702.08734.

[10] Andrew McCallum, Kamal Nigam, and Lyle H. Ungar. Efficient clustering of high-dimensional data sets with application to reference matching. In Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), pages 169–178, 2000. https://doi.org/10.1145/347090.347123. Data set (Cora): https://www.openicpsr.org/openicpsr/project/100859/version/V1/view and https://hpi.de/naumann/projects/repeatability/datasets/cora-dataset.html.

[11] Saravanan Thirumuruganathan, Han Li, Nan Tang, Mourad Ouzzani, Yash Govind, Derek Paulsen, Glenn Fung, and AnHai Doan. Deep learning for blocking in entity matching. Proceedings of the VLDB Endowment, 14(11):2459–2472, 2021. https://doi.org/10.14778/3476249.3476294.

[12] Wei Zhang, Hao Wei, Bunyamin Sisman, Xin Luna Dong, Christos Faloutsos, and David Page. AutoBlock: A hands-off blocking framework for entity matching. In Proceedings of the 13th ACM International Conference on Web Search and Data Mining (WSDM), pages 744–752, 2020. https://doi.org/10.1145/3336191.3371813.

[13] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In Proceedings of the 2018 International Conference on Management of Data (SIGMOD), pages 19–34, 2018. https://doi.org/10.1145/3183713.3196926.

[14] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, and Kostas Stefanidis. An overview of end-to-end entity resolution for big data. ACM Computing Surveys, 53(6):127:1–127:42, 2021. https://doi.org/10.1145/3418896.

[15] Mauricio Sadinle and Stephen E. Fienberg. A generalized Fellegi–Sunter framework for multiple record linkage with application to homicide record systems. Journal of the American Statistical Association, 108(502):651–660, 2013. https://arxiv.org/abs/1205.3217.

[16] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. Deep entity matching with pre-trained language models. Proceedings of the VLDB Endowment, 14(1):50–60, 2020. https://doi.org/10.14778/3421424.3421431.

[17] Ralph Peeters, Aaron Steiner, and Christian Bizer. Entity matching using large language models. arXiv preprint arXiv:2310.11244, 2023. https://arxiv.org/abs/2310.11244.

[18] North Carolina State Board of Elections. Voter registration data (public data download). https://www.ncsbe.gov/results-data/voter-registration-data.

[19] William E. Winkler. Improved decision rules in the Fellegi–Sunter model of record linkage. In Proceedings of the Section on Survey Research Methods, American Statistical Association, pages 274–279, 1993.

[20] William E. Winkler. Frequency-based matching in the Fellegi–Sunter model of record linkage. Statistical Research Division, U.S. Bureau of the Census, Research Report Series RR2000/06, 2000. https://www.census.gov/library/working-papers/2000/adrm/rr2000-06.html.

[21] William E. Winkler. Overview of record linkage and current research directions. Statistical Research Division, U.S. Bureau of the Census, Research Report Series RRS2006/02, 2006. https://www.census.gov/library/working-papers/2006/adrm/rrs2006-02.html.

[22] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In Proceedings of the 23rd International Conference on Machine Learning (ICML), pages 233–240, 2006. https://doi.org/10.1145/1143844.1143874.

[23] Peter Christen. Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Springer, Data-Centric Systems and Applications, 2012. https://doi.org/10.1007/978-3-642-31164-2.

[24] Denis Robert. Time decay in Fellegi–Sunter record linkage: A literature survey. Technical note, Independent Researcher, 2026.

[25] Robin Linacre. Comment on Splink issue #3240 concerning piecewise temporal decay via comparison levels. GitHub, moj-analytical-services/splink, 2026. https://github.com/moj-analytical-services/splink/issues/3240#issuecomment-5309218620.

[26] Denis Robert. Batch deduplication and insertion-time prevention with a shared vector canopy index. Technical report, Independent Researcher, 2026.